

CONTENTS

- A Practical Roadmap for Whole-Exome Sequencing Data Analysis
 - 0. Environment, conventions and our roadmap
 - 1. FASTQ files
 - 2. Read quality control with fastp
 - 3. Alignment with BWA-MEM
 - 4. Mark PCR duplicates
 - 5. Base quality score recalibration
 - 6. Germline variant calling
 - 7. Variant quality score recalibration
 - 8. Functional annotation with ANNOVAR
 - 9. Somatic TMB using matched normal
 - 10. CNV detection with CNVkit
 - 11. Interpretation checklist
 - 12. File summary
 - 13. Summary

A Practical Roadmap for Whole-Exome Sequencing Data Analysis

Jiahao Ma (jiahaoma@stu.pku.edu.cn), Zexian Zeng (zexianzeng@pku.edu.cn)

Center of Life Sciences & Center for Quantitative Biology

Academy for Advanced Interdisciplinary Studies, Peking University, Beijing, China

Thanks to Yufeng He for his previous work on teaching roadmap.

[Lab website](#)

Today, we celebrate the revelation of the first draft of the human book of life.
— Francis Collins

0. Environment, conventions and our roadmap

“工欲善其事，必先利其器。”
——《论语·卫灵公》

0.1 Required tools

Tool	Main purpose
<code>fastp</code>	Read trimming, adapter removal, per-base quality filtering, QC report
<code>bwa</code>	Alignment of reads to a reference genome
<code>samtools</code>	SAM/BAM conversion, sorting, indexing, coverage and flag statistics
<code>bcftools</code>	VCF viewing, filtering, compression and indexing
<code>GATK4</code>	Duplicate marking, BQSR, germline and somatic variant calling, VQSR
<code>ANNOVAR</code>	Functional annotation of variants
<code>CNVkit</code>	Copy-number variation detection from targeted sequencing data

On the teaching cluster, log in and use the shared `wes` environment. The shared directory is read-only for teaching purposes; write all personal outputs under `/gpfs1/home/$USER`.

```
ssh CCIPKUHPC_node03
```

Add the shared Miniconda initialization and workflow variables to `/gpfs1/home/$USER/.bashrc`:

```
cat >> ~/.bashrc <<'EOF'

# WES teaching-cluster initialization
export SHARE=/lustre1/share
export MINICONDA=$SHARE/miniconda3
source "$MINICONDA/etc/profile.d/conda.sh"
EOF

source ~/.bashrc
conda activate wes
```

Then add the following variables to your current shell:

```
export SHARE=/lustre1/share
export REF=$SHARE/references
export TARGETS=$REF/S07604514_AllTracks_V6_60_hg38.bed
export REFERENCE=$REF/hg38.fa
export DBSNP=$REF/dbsnp_138.hg38.vcf
export INDELS=$REF/Mills_and_1000G_gold_standard.indels.hg38.vcf
export HIGHCONF=$REF/1000G_phase1.snps.high_confidence.hg38.vcf
```

```

export HAPMAP=$REF/hapmap_3.3.hg38.vcf
export OMNI=$REF/1000G_omni2.5.hg38.vcf
export ANNOVAR=$SHARE/annovar_new
export HUMANDB=$ANNOVAR/humandb
export PATH=$ANNOVAR:$PATH
export DATA=$SHARE/analysis/0_preprocess
export BASE=/lustre1/user/$USER/wes_run_cluster_$(date +%Y%m%d) # Caution: if you want to
continue the previous run, please change this to exact previous run path, e.g.,
/gpfs1/home/$USER/wes_run_cluster_20260911
export OUT=$BASE

```

Attention: If you want to run the analysis in a new shell (e.g., a new ssh terminal), you need to export these variables again.

The required executables are provided by the shared environment, you can run the following commands to check the versions:

```

fastp --version
bwa 2>&1 | head -n 3
samtools --version | head -n 1
bcftools --version | head -n 1
gatk --version
cnvkit.py version
table_annovar.pl --version

```

0.2 Directory convention

The examples use the following directories. Each student writes under their own home directory and reads shared data and references:

```

mkdir -p
"$OUT"/{0_fastq,1_alignment/OC,1_alignment/PBMC,2_variant_call,3_annotation,4_cnv,5_somatic_t
mb}
mkdir -p "$OUT"/{log,resources}

```

For this teaching run, the raw inputs are copied from the preloaded user analysis directory rather than written into `/lustre1/share/analysis` :

```

ln -s "$DATA/OC/OC_R1.fq.gz" "$OUT/0_fastq/OC_R1.fq.gz"
ln -s "$DATA/OC/OC_R2.fq.gz" "$OUT/0_fastq/OC_R2.fq.gz"
ln -s "$DATA/PBMC/blood_R1.fq.gz" "$OUT/0_fastq/PBMC_R1.fq.gz"
ln -s "$DATA/PBMC/blood_R2.fq.gz" "$OUT/0_fastq/PBMC_R2.fq.gz"

```

The shared ANNOVAR installation is:

```

ANNOVAR=/lustre1/share/annovar/annovar
HUMANDB=$ANNOVAR/humandb

```

The two example samples are:

OC: ovarian-cancer tissue sample, treated as the tumor sample
PBMC: matched blood sample, treated as the normal sample

0.3 Reference genome

Use a single consistent genome build throughout the workflow. The shared teaching references use GRCh38/hg38:

\$REF/hg38.fa
\$REF/hg38.dict
\$REF/hg38.fa.fai

GATK also requires a sequence dictionary. They are already present in the shared reference directory, so verify rather than regenerate them:

```
ls -lh "$REFERENCE"{,.fai,.amb,.ann,.bwt,.pac,.sa} "$REF/hg38.dict"
```

0.4 Roadmap

WES Analysis Pipeline Overview

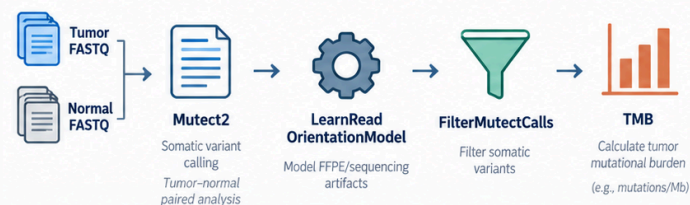
1. Germline SNP/Indel Variant Calling Pipeline (WES)

From raw reads to high-confidence annotated variants



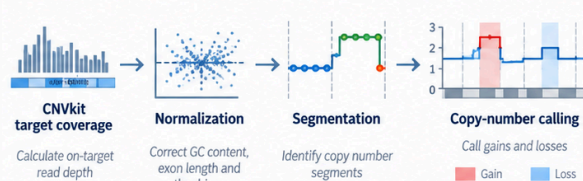
2. Tumor-Normal Somatic TMB Analysis (Extension)

Identify somatic variants and calculate TMB



3. WES CNV Detection (Extension)

Infer copy number alterations from WES data



1. FASTQ files

FASTQ stores biological sequence plus a per-base quality score. A paired-end read is stored in two files, usually named **R1** and **R2**. Each sequence occupies four lines:

```
@instrument:run:flowcell:lane:tile:x:y read1:N:0:INDEX1+INDEX2
ACGTACGTACGT ...
+
IIIIIIHHHHFFFF ...
```

The first line is the read identifier. The second line is the nucleotide sequence. The third line is a separator. The fourth line contains Phred quality scores encoded as ASCII characters.

The Phred score is:

```
Q = -10 * log10(P_error)
P_error = 10^(-Q/10)
```

Examples:

Phred score	Probability of error	Accuracy
Q10	0.1	90%
Q20	0.01	99%
Q30	0.001	99.9%
Q40	0.0001	99.99%

FASTQ record and Phred quality scores

Example FASTQ record (one read)

ID @NB501726:6:H7W2LAFX:1:11101:10000:1000 1:N:0:1
Sequence ACGTTGCAATGCCGTAACCTGAACTTGA
+ +
Quality ASCII IIIIIIHHHHHHGGGGGGFFFFFFFEEEEEDDDC

A FASTQ record consists of four lines: a read identifier (ID), the nucleotide sequence, a '+' separator, and a string of ASCII characters encoding the Phred quality score for each base.

Phred quality score

The Phred quality score (Q) indicates the probability that a base call is incorrect (P_{error}). Higher scores mean higher confidence (greater accuracy).

Phred score from error probability:

$$Q = -10 \log_{10}(P_{\text{error}})$$

Error probability from Phred score:

$$P_{\text{error}} = 10^{(-Q/10)}$$

Common Phred scores and their interpretation

Q score	Probability of error (P_{error})	Sequencing accuracy ($1 - P_{\text{error}}$)
Q10	1×10^{-1} (0.1)	90%
Q20	1×10^{-2} (0.01)	99%
Q30	1×10^{-3} (0.001)	99.9%
Q40	1×10^{-4} (0.0001)	99.99%

1.1 Inspect FASTQ

View the first eight lines:

```
zcat $OUT/0_fastq/OC_R1.fq.gz | head -n 8
```

Count reads in a FASTQ file:

```
zcat $OUT/0_fastq/OC_R1.fq.gz | awk 'END {print NR/4}'
```

2. Read quality control with fastp

"The first principle is that you must not fool yourself—and you are the easiest person to fool."
— Richard Feynman

Raw reads may contain adapters, low-quality tails, poly-G artifacts, and very short fragments. `fastp` removes these problematic reads or bases and produces HTML/JSON QC reports.

The algorithmic logic is:

1. Detect adapters from overlapping read pairs or known adapter sequences.
2. Trim low-quality bases from read ends.
3. Remove reads that are too short after trimming.
4. Summarize per-base quality, GC content, adapter contamination, duplication, and insertion-size distributions.

2.1 Create QC output directories

```
mkdir -p "$OUT/0_fastq/OC" "$OUT/0_fastq/PBMC"
```

2.2 Run fastp

For OC:

```
fastp \
-i "$OUT/0_fastq/OC_R1.fq.gz" \
-I "$OUT/0_fastq/OC_R2.fq.gz" \
-o "$OUT/0_fastq/OC_R1.clean.fq.gz" \
-O "$OUT/0_fastq/OC_R2.clean.fq.gz" \
--detect_adapter_for_pe \
--cut_right \
--cut_right_window_size 4 \
--cut_right_mean_quality 20 \
--qualified_quality_phred 20 \
--unqualified_percent_limit 40 \
--n_base_limit 5 \
--length_required 50 \
--thread 8 \
--html "$OUT/0_fastq/OC/OC.fastp.html" \
--json "$OUT/0_fastq/OC/OC.fastp.json"
```

For PBMC:

```
fastp \
-i "$OUT/0_fastq/PBMC_R1.fq.gz" \
-I "$OUT/0_fastq/PBMC_R2.fq.gz" \
-o "$OUT/0_fastq/PBMC_R1.clean.fq.gz" \
-O "$OUT/0_fastq/PBMC_R2.clean.fq.gz" \
--detect_adapter_for_pe \
--cut_right \
--cut_right_window_size 4 \
--cut_right_mean_quality 20 \
--qualified_quality_phred 20 \
--unqualified_percent_limit 40 \
--n_base_limit 5 \
```

```
--length_required 50 \  
--thread 8 \  
--html "$OUT/0_fastq/PBMC/blood.fastp.html" \  
--json "$OUT/0_fastq/PBMC/blood.fastp.json"
```

Important fastp metrics:

Metric	Interpretation
<code>before_filtering</code> / <code>after_filtering</code>	Raw and cleaned read counts
<code>q20_rate</code> , <code>q30_rate</code>	Fraction of bases with Q20 or Q30
<code>adapter_cutting</code>	Adapter contamination
<code>duplication</code>	Apparent duplicate rate at FASTQ level
<code>insert_size</code>	Library insert-size distribution

3. Alignment with BWA-MEM

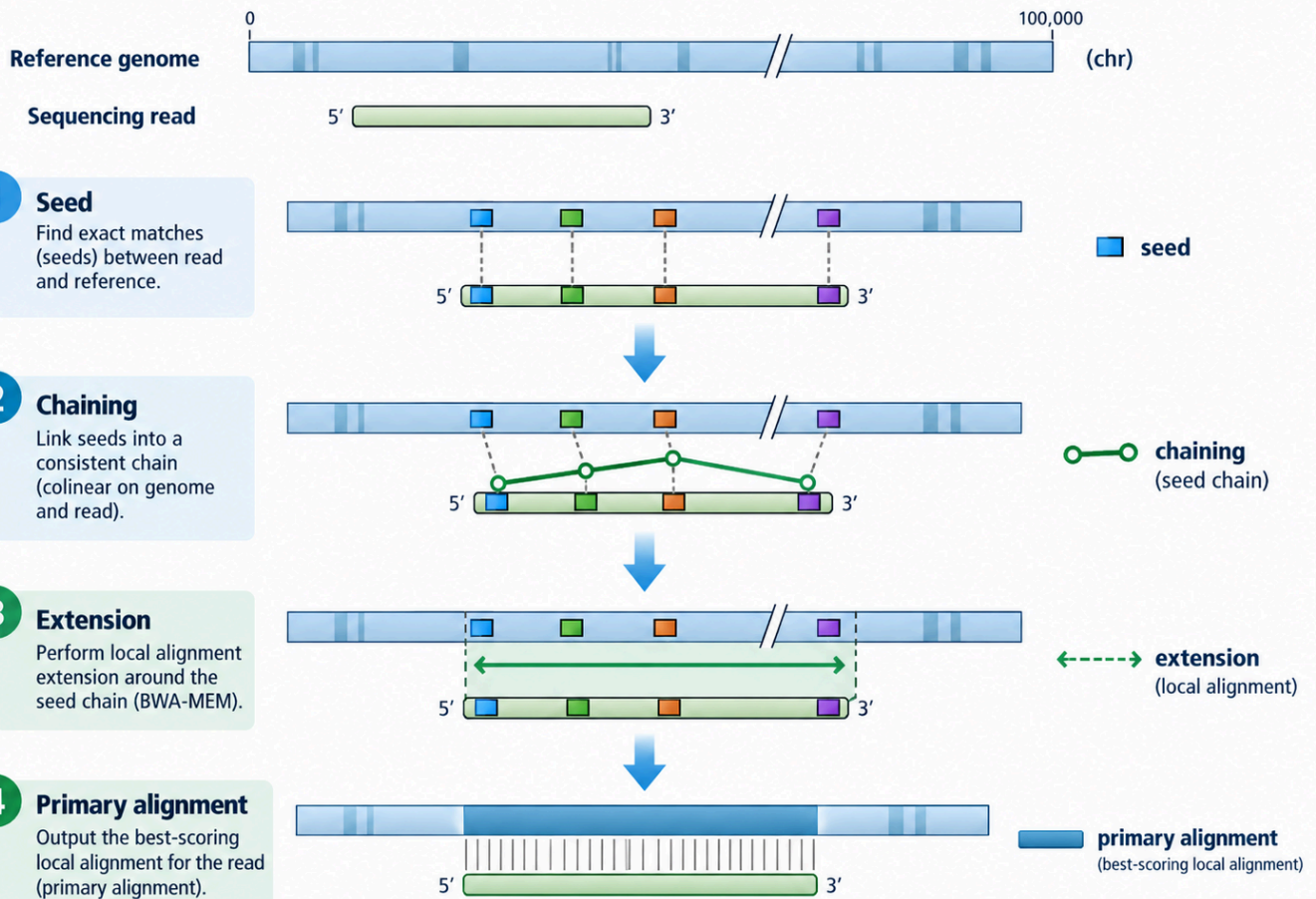
"Science is a voyage of exploration into the unknown."
— Francis Collins

Alignment places each read onto the reference genome. BWA-MEM is designed for Illumina reads roughly 70 bp or longer.

BWA-MEM uses a seed-and-extend strategy:

1. Find long exact or near-exact matches between read and reference.
2. Chain seeds that appear collinear in both read and reference.
3. Extend chains with Smith-Waterman-like local alignment.
4. Score candidate alignments and choose the primary alignment.
5. Report supplementary alignments for split reads.

BWA-MEM seed-and-extend



3.1 Alignment

For OC:

```
bwa mem \
-t 8 \
-M \
-R "@RG\tID:OC\tPL:illumina\tSM:OC\tLB:WES_OC" \
"$REFERENCE" \
"$OUT/0_fastq/OC_R1.clean.fq.gz" \
"$OUT/0_fastq/OC_R2.clean.fq.gz" \
> "$OUT/1_alignment/OC/OC.sam"
```

For PBMC:

```
bwa mem \
-t 8 \
-M \
-R "@RG\tID:PBMC\tPL:illumina\tSM:blood\tLB:WES_PBMC" \
"$REFERENCE" \
"$OUT/0_fastq/PBMC_R1.clean.fq.gz" \
"$OUT/0_fastq/PBMC_R2.clean.fq.gz" \
> "$OUT/1_alignment/PBMC/blood.sam"
```

The read group is important:

Field	Meaning
ID	Unique read-group identifier
PL	Sequencing platform
SM	Sample name
LB	DNA library name

The sample name **SM** must be correct. GATK uses it to distinguish samples during joint genotyping or tumor-normal calling.

3.2 SAM to BAM

SAM is text; BAM is binary. BAM is smaller and faster for software to read.

```
samtools view -bS -@ 8 \  
"$OUT/1_alignment/OC/OC.sam" \  
> "$OUT/1_alignment/OC/OC.bam"  
  
samtools view -bS -@ 8 \  
"$OUT/1_alignment/PBMC/blood.sam" \  
> "$OUT/1_alignment/PBMC/blood.bam"
```

3.3 Sort and index BAM

Variant callers require coordinate-sorted BAM files.

```
samtools sort -@ 8 -m 4G \  
"$OUT/1_alignment/OC/OC.bam" \  
-o "$OUT/1_alignment/OC/OC_sorted.bam"  
  
samtools sort -@ 8 -m 4G \  
"$OUT/1_alignment/PBMC/blood.bam" \  
-o "$OUT/1_alignment/PBMC/blood_sorted.bam"  
  
samtools index -@ 8 "$OUT/1_alignment/OC/OC_sorted.bam"  
samtools index -@ 8 "$OUT/1_alignment/PBMC/blood_sorted.bam"
```

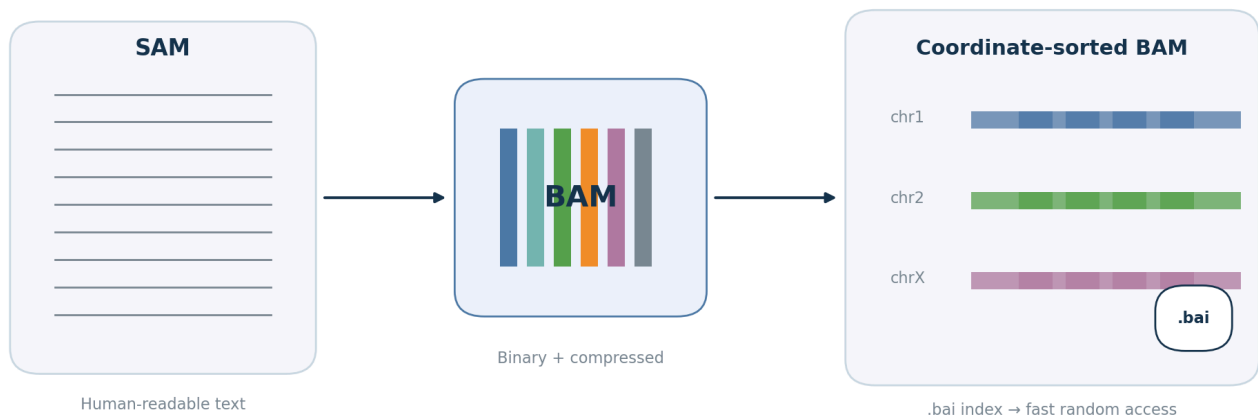
3.4 Alignment metrics

```
samtools flagstat -@ 8 "$OUT/1_alignment/OC/OC_sorted.bam" \  
> "$OUT/1_alignment/OC/OC.flagstat.txt"  
  
samtools flagstat -@ 8 "$OUT/1_alignment/PBMC/blood_sorted.bam" \  
> "$OUT/1_alignment/PBMC/blood.flagstat.txt"
```

Key metrics:

Metric	Desired pattern
Mapped reads	Usually >95% for WES
Properly paired	High, typically >90%
Duplicate rate	Library-dependent, should not be extreme
Mismatch rate	Should be low after QC

SAM, BAM, coordinate sorting and indexing



4. Mark PCR duplicates

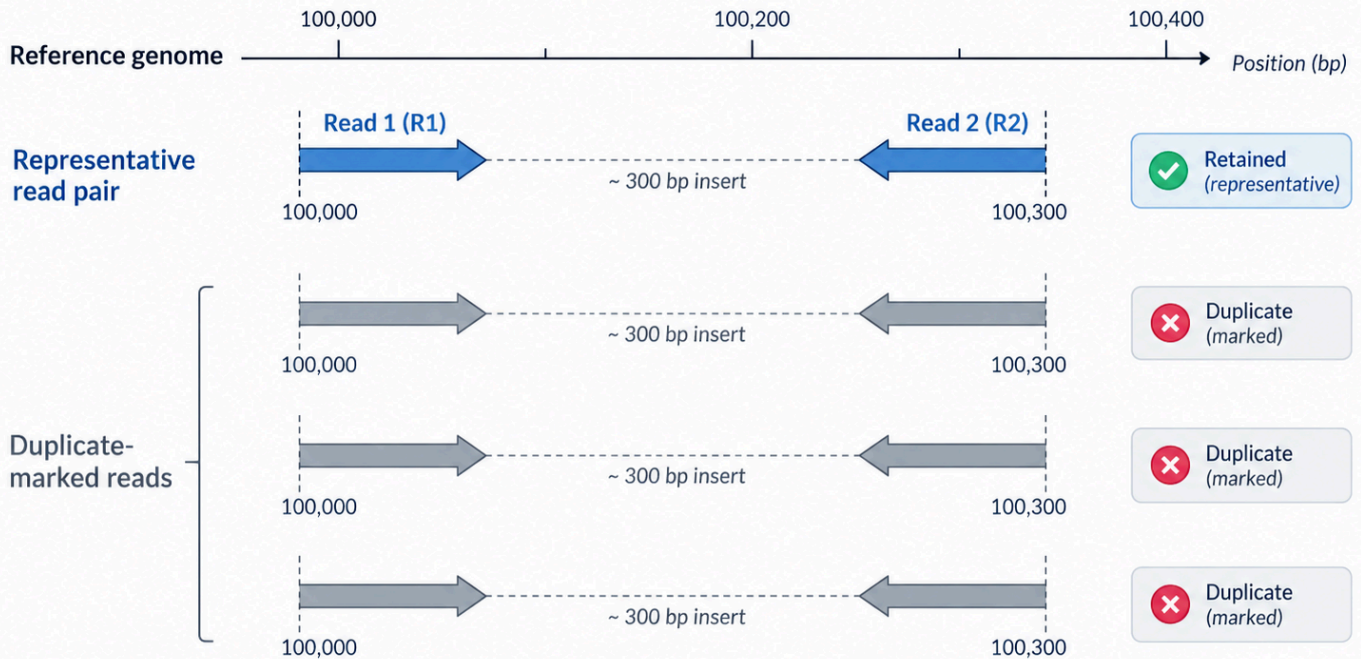
Duplicate read pairs often arise from PCR amplification of the same original DNA fragment. If the same fragment is sequenced many times, those reads are not independent observations. Treating them as independent can inflate confidence in a variant.

GATK `MarkDuplicates` identifies duplicate read pairs primarily by:

1. Unclipped alignment start.
2. Unclipped alignment end.
3. Read-pair orientation.
4. Mate position.

It then marks all but one read pair as duplicates. The representative read is chosen using base quality and other criteria. The BAM records are not usually removed; they are flagged.

Marking PCR duplicates



Rule:

Read pairs with the same start and end coordinates and the same orientation are considered PCR duplicates (typically arising from amplification) and all but one are marked as duplicates.

4.1 Mark duplicates

For OC:

```
gatk --java-options "-Xmx8g" MarkDuplicates \  
-I "$OUT/1_alignment/OC/OC_sorted.bam" \  
-O "$OUT/1_alignment/OC/OC_sorted.markdup.bam" \  
-M "$OUT/1_alignment/OC/OC_markdup_metrics.txt" \  
--CREATE_INDEX true
```

For PBMC:

```
gatk --java-options "-Xmx8g" MarkDuplicates \  
-I "$OUT/1_alignment/PBMC/blood_sorted.bam" \  
-O "$OUT/1_alignment/PBMC/blood_sorted.markdup.bam" \  
-M "$OUT/1_alignment/PBMC/blood_markdup_metrics.txt" \  
--CREATE_INDEX true
```

4.2 Inspect duplicate metrics

```
grep -A 6 '^LIBRARY' "$OUT/1_alignment/OC/OC_markdup_metrics.txt"
```

Important columns:

Column	Meaning
READ_PAIRS_EXAMINED	Number of read pairs used for duplicate analysis
READ_PAIR_DUPLICATES	Duplicate read pairs
PERCENT_DUPLICATION	Fraction of data marked as duplicates

5. Base quality score recalibration

FASTQ quality scores are the sequencer's initial estimates of per-base error probability. These estimates can be systematically biased by sequencing chemistry, cycle, read position, and sequence context.

BQSR does not change the called bases. It changes how much the caller trusts each base.

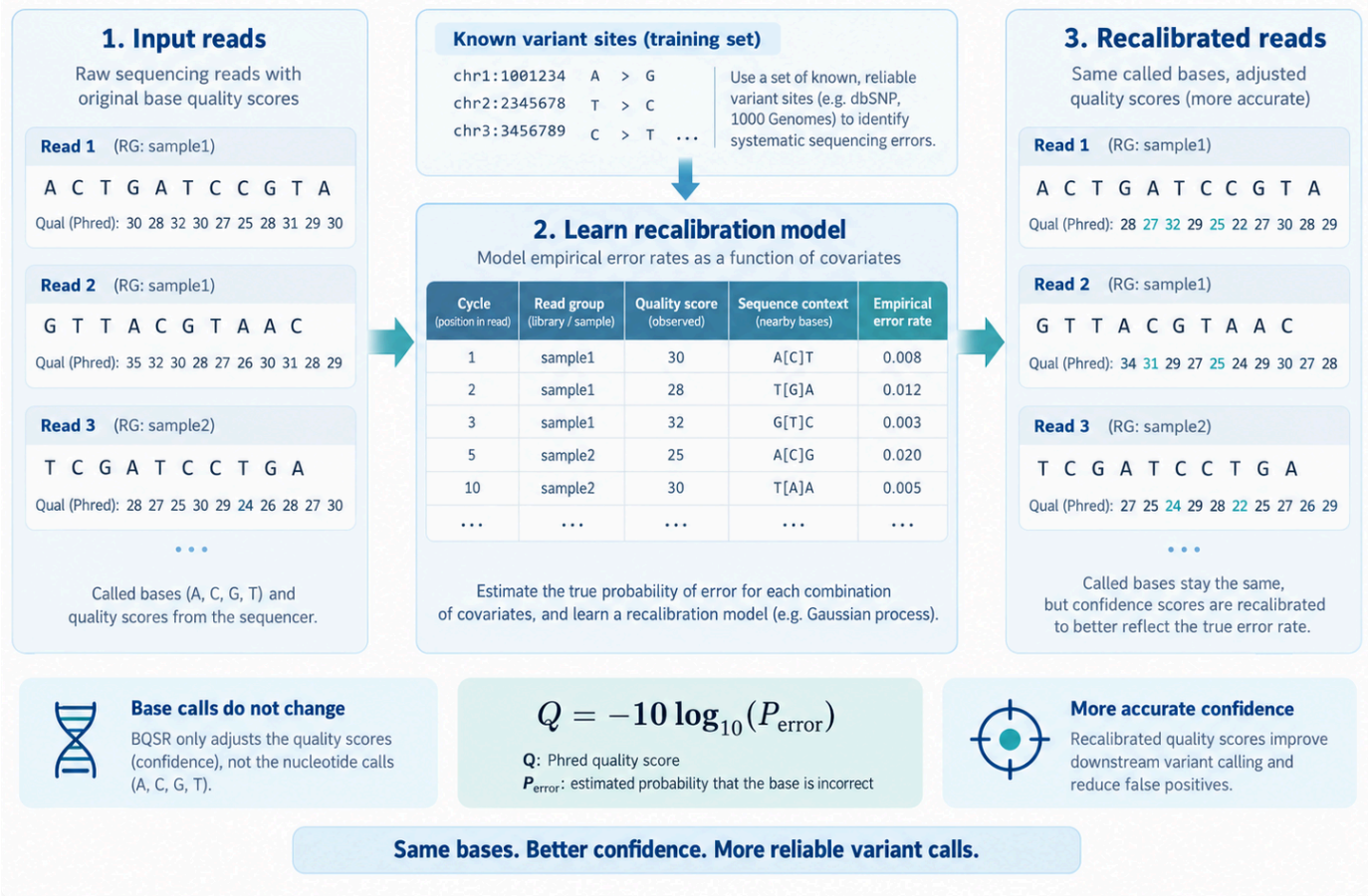
The algorithm uses known polymorphic sites to distinguish real variation from technical error. It then models systematic errors using covariates such as:

```
read group
quality score
machine cycle
sequence context
```

For a base with reported quality `Q_reported`, the empirical error rate among similar bases is used to infer an adjusted quality `Q_adjusted`.

Base quality score recalibration (BQSR)

Learn systematic errors from known variants and recalibrate base quality scores to better reflect the true probability of error.



5.1 Learn the recalibration model

```
gatk --java-options "-Xmx20g -XX:ParallelGCThreads=4" BaseRecalibrator \
-R "$REFERENCE" \
-I "$OUT/1_alignment/OC/OC_sorted.markdup.bam" \
--known-sites "$DBSNP" \
--known-sites "$INDELS" \
--known-sites "$HIGHCONF" \
-O "$OUT/1_alignment/OC/OC_recal_data.table"

gatk --java-options "-Xmx20g -XX:ParallelGCThreads=4" BaseRecalibrator \
-R "$REFERENCE" \
-I "$OUT/1_alignment/PBMC/blood_sorted.markdup.bam" \
--known-sites "$DBSNP" \
--known-sites "$INDELS" \
--known-sites "$HIGHCONF" \
-O "$OUT/1_alignment/PBMC/blood_recal_data.table"
```

Resource roles:

Resource	Use in BQSR
dbSNP	Known variant sites; excluded from error estimation

Resource	Use in BQSR
Mills/1000G indels	High-confidence indel truth sites
1000G high-confidence SNPs	High-confidence SNP truth sites

5.2 Apply recalibration

```
gatk --java-options "-Xmx12g -XX:ParallelGCThreads=4" ApplyBQSR \
-R "$REFERENCE" \
-I "$OUT/1_alignment/OC/OC_sorted.markdup.bam" \
--bqsr-recal-file "$OUT/1_alignment/OC/OC.recal_data.table" \
-O "$OUT/1_alignment/OC/OC_sorted.markdup.BQSR.bam"

gatk --java-options "-Xmx12g -XX:ParallelGCThreads=4" ApplyBQSR \
-R "$REFERENCE" \
-I "$OUT/1_alignment/PBMC/blood_sorted.markdup.bam" \
--bqsr-recal-file "$OUT/1_alignment/PBMC/blood.recal_data.table" \
-O "$OUT/1_alignment/PBMC/blood_sorted.markdup.BQSR.bam"
```

6. Germline variant calling

"The purpose of computing is insight, not numbers."

— Richard Hamming

The main short variants are:

Variant type	Definition
SNP	Single-nucleotide polymorphism
SNV	Single-nucleotide variant relative to reference
Insertion	Added sequence relative to reference
Deletion	Missing sequence relative to reference
MNP	Multiple adjacent substituted bases

6.1 HaplotypeCaller algorithm

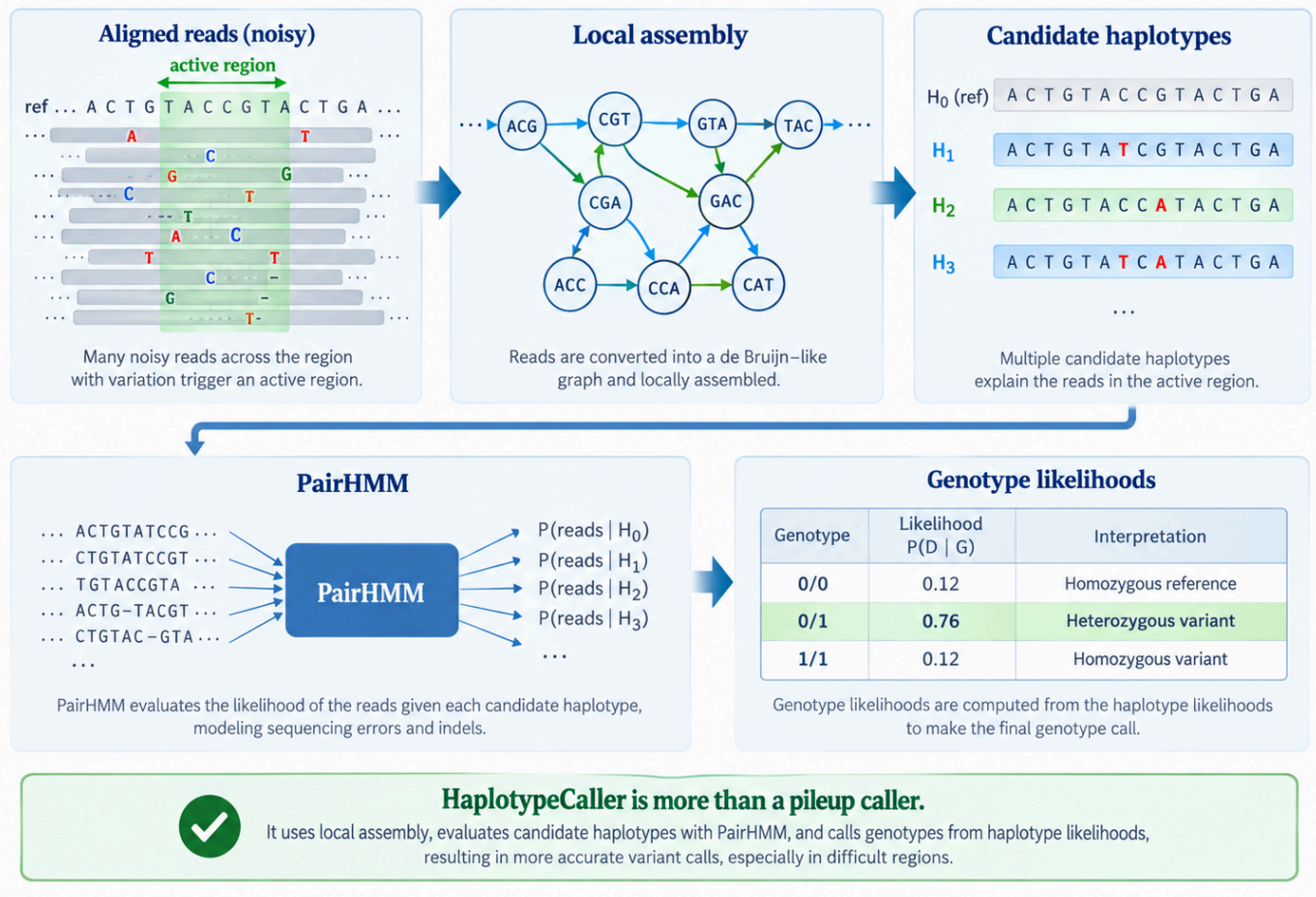
GATK HaplotypeCaller is not a pileup-only caller. Its key idea is local haplotype reconstruction:

1. Identify active regions with variation evidence.
2. Locally reassemble reads into candidate haplotypes.
3. Build a de Bruijn-like graph from k-mers.

4. Align candidate haplotypes to the reference.
5. Use PairHMM to compute read-haplotype likelihoods.
6. Use Bayesian genotype likelihoods to choose the most likely genotype.

PairHMM handles alignment uncertainty, especially near indels and homopolymers.

HaplotypeCaller local assembly



6.2 Call GVCFs

For OC:

```
gatk --java-options "-Xmx12g -XX:ParallelGCThreads=4" HaplotypeCaller \
  -ERC GVCF \
  -R "$REFERENCE" \
  -I "$OUT/1_alignment/OC/OC_sorted.markdup.BQSR.bam" \
  -D "$DBSNP" \
  -L "$TARGETS" \
  --native-pair-hmm-threads 8 \
  -O "$OUT/1_alignment/OC/OC.g.vcf"
```

For PBMC:

```
gatk --java-options "-Xmx12g -XX:ParallelGCThreads=4" HaplotypeCaller \
  -ERC GVCF \
  -R "$REFERENCE" \
  -I "$OUT/1_alignment/PBMC/blood_sorted.markdup.BQSR.bam" \
  -D "$DBSNP" \
  -L "$TARGETS" \
  --native-pair-hmm-threads 8 \
  -O "$OUT/1_alignment/PBMC/blood.g.vcf"
```

GVCF output includes both variant sites and confident hom-ref sites, which is useful for joint genotyping.

6.3 Combine and genotype GVCFs

```
gatk --java-options "-Xmx16g -XX:ParallelGCThreads=4" CombineGVCFs \
  -R "$REFERENCE" \
  -V "$OUT/1_alignment/OC/OC.g.vcf" \
  -V "$OUT/1_alignment/PBMC/blood.g.vcf" \
  -O "$OUT/2_variant_call/combined.g.vcf"

gatk --java-options "-Xmx16g -XX:ParallelGCThreads=4" GenotypeGVCFs \
  -R "$REFERENCE" \
  -V "$OUT/2_variant_call/combined.g.vcf" \
  -D "$DBSNP" \
  -O "$OUT/2_variant_call/OC_blood_variants.vcf"
```

6.4 VariantAnnotator

Some teaching workflows explicitly run VariantAnnotator before VQSR:

```
gatk --java-options "-Xmx12g -XX:ParallelGCThreads=4" VariantAnnotator \
  -R "$REFERENCE" \
  -I "$OUT/1_alignment/OC/OC_sorted.markdup.BQSR.bam" \
  -V "$OUT/2_variant_call/OC_blood_variants.vcf" \
  -D "$DBSNP" \
  -O "$OUT/2_variant_call/OC_blood_variants.anno.vcf"
```

In newer GATK4 workflows, HaplotypeCaller already emits many annotations, so this step may be redundant.

6.5 VCF format

A VCF record has eight mandatory columns:

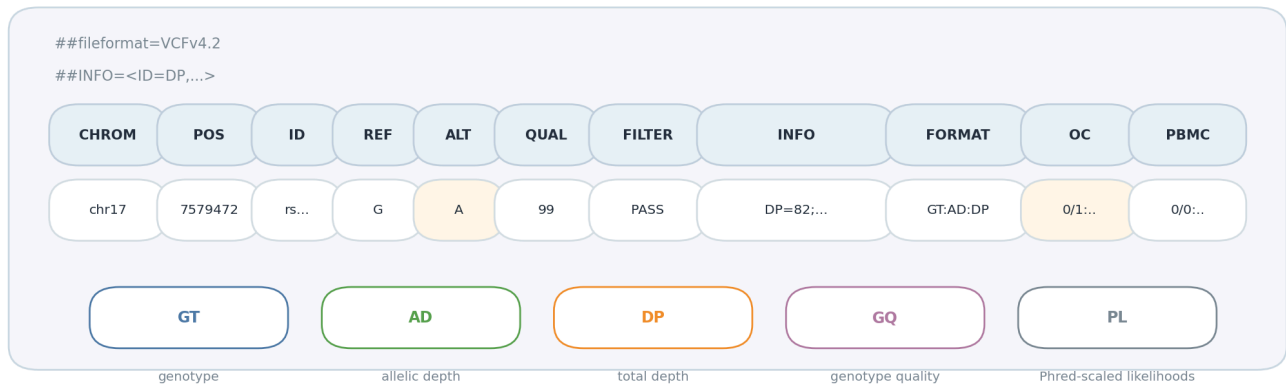
```
CHROM POS ID REF ALT QUAL FILTER INFO
```

After genotype columns, the common FORMAT fields are:

Field	Meaning
GT	Genotype, e.g. <code>0/0</code> , <code>0/1</code> , <code>1/1</code>

Field	Meaning
AD	Allelic depth for reference and alternate alleles
DP	Total depth
GQ	Genotype quality
PL	Phred-scaled likelihoods for possible genotypes

Variant Call Format (VCF)



7. Variant quality score recalibration

"All models are wrong, but some are useful."
— George E. P. Box

VQSR learns what true variants and likely artifacts look like in annotation space.

For SNPs, typical annotations are:

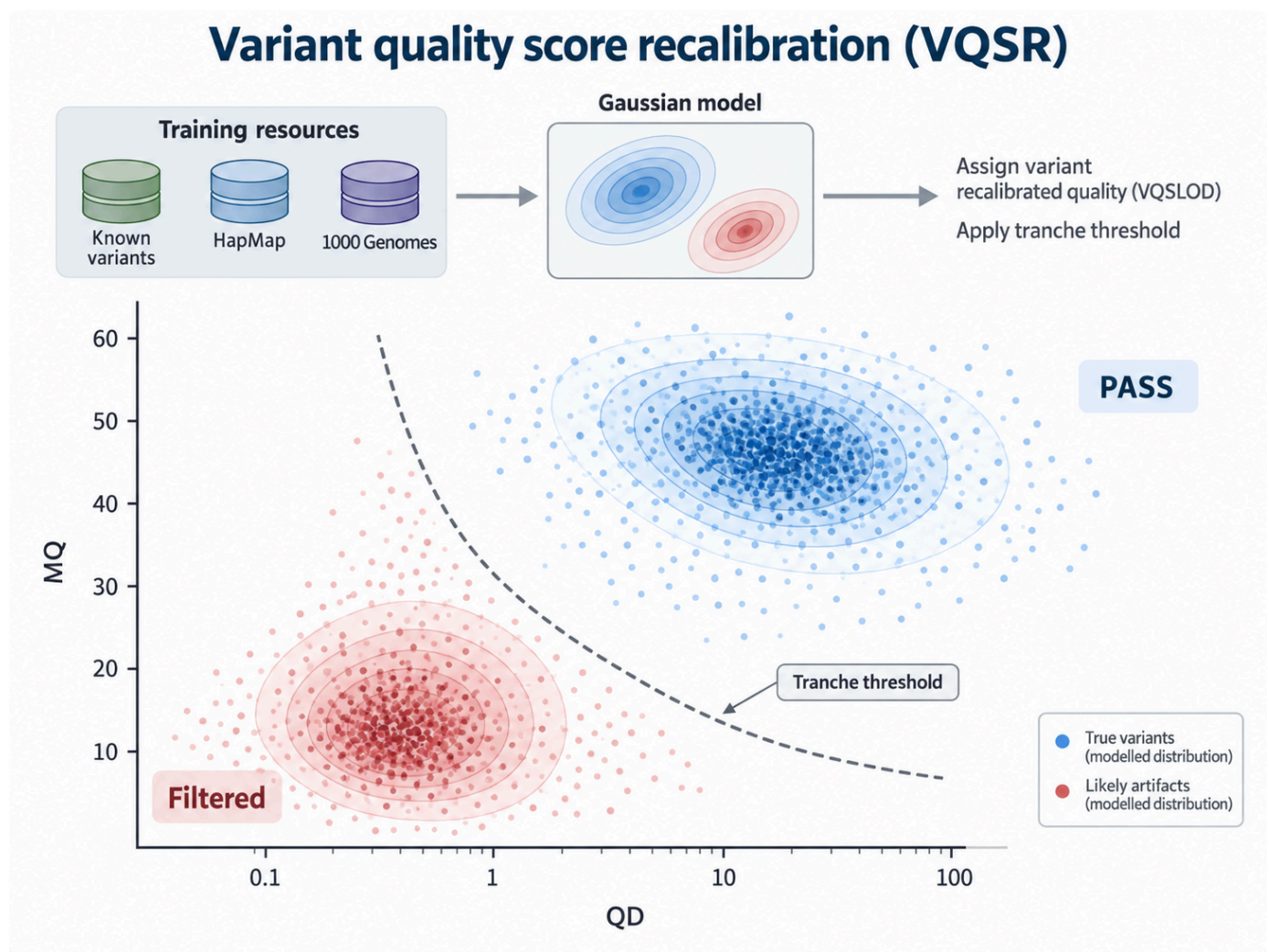
Annotation	Intuition
QD	Variant confidence divided by depth
MQ	Mapping quality
MQRankSum	Mapping-quality difference between alt and ref reads
ReadPosRankSum	Alt-vs-ref read-position bias
FS	Fisher strand bias
SOR	Symmetric odds ratio strand bias

For indels, GATK often uses a smaller annotation set such as `QD` , `ReadPosRankSum` , `FS` , and `SOR` .

The VariantRecalibrator fits a Gaussian mixture model:

1. High-confidence truth resources define likely true variants.
2. Known but non-truth resources define known variants.
3. Novel variants are treated as possible false positives.
4. The model scores each variant by similarity to true variation.
5. Tranches define sensitivity levels, for example 99.5% SNP truth sensitivity.

ApplyVQSR labels variants as `PASS` or filtered according to the chosen tranche.



7.1 SNP VQSR

```
gatk --java-options "-Xmx20g -XX:ParallelGCThreads=4" VariantRecalibrator \  
-R "$REFERENCE" \  
-V "$OUT/2_variant_call/OC_blood_variants.vcf" \  
--resource:hapmap,known=false,training=true,truth=true,prior=15.0 "$HAPMAP" \  
--resource:omni,known=false,training=true,truth=false,prior=12.0 "$OMNI" \  
--resource:1000G,known=false,training=true,truth=false,prior=10.0 "$HIGHCONF" \  
--resource:dbsnp,known=true,training=false,truth=false,prior=2.0 "$DBSNP" \  
-an QD -an MQ -an MQRankSum -an ReadPosRankSum -an FS -an SOR \  

```

```

-mode SNP \
-O "$OUT/2_variant_call/OC_blood.snps.recal" \
--tranches-file "$OUT/2_variant_call/OC_blood.snps.tranches" \
--rscript-file "$OUT/2_variant_call/OC_blood.snps.plots.R" \
--dont-run-rscript

gatk --java-options "-Xmx16g -XX:ParallelGCThreads=4" ApplyVQSR \
-R "$REFERENCE" \
-V "$OUT/2_variant_call/OC_blood_variants.vcf" \
-O "$OUT/2_variant_call/OC_blood.snps.VQSR.vcf" \
--truth-sensitivity-filter-level 99.5 \
--tranches-file "$OUT/2_variant_call/OC_blood.snps.tranches" \
--recal-file "$OUT/2_variant_call/OC_blood.snps.recal" \
--mode SNP

```

7.2 INDEL VQSR

```

gatk --java-options "-Xmx20g -XX:ParallelGCThreads=4" VariantRecalibrator \
-R "$REFERENCE" \
-V "$OUT/2_variant_call/OC_blood_variants.vcf" \
--resource:mills,known=false,training=true,truth=true,prior=12.0 "$INDELS" \
--resource:dbsnp,known=true,training=false,truth=false,prior=2.0 "$DBSNP" \
--an QD --an ReadPosRankSum --an FS --an SOR \
-mode INDEL \
--max-gaussians 4 \
-O "$OUT/2_variant_call/OC_blood.indel.recal" \
--tranches-file "$OUT/2_variant_call/OC_blood.indel.tranches" \
--rscript-file "$OUT/2_variant_call/OC_blood.indel.plots.R" \
--dont-run-rscript

gatk --java-options "-Xmx16g -XX:ParallelGCThreads=4" ApplyVQSR \
-R "$REFERENCE" \
-V "$OUT/2_variant_call/OC_blood_variants.vcf" \
-O "$OUT/2_variant_call/OC_blood.indel.VQSR.vcf" \
--truth-sensitivity-filter-level 99.0 \
--tranches-file "$OUT/2_variant_call/OC_blood.indel.tranches" \
--recal-file "$OUT/2_variant_call/OC_blood.indel.recal" \
--mode INDEL

```

7.3 Merge SNP and indel outputs

```

gatk --java-options "-Xmx12g -XX:ParallelGCThreads=4" MergeVcfs \
-O "$OUT/2_variant_call/OC_blood.all.VQSR.vcf" \
-I "$OUT/2_variant_call/OC_blood.indel.VQSR.vcf" \
-I "$OUT/2_variant_call/OC_blood.snps.VQSR.vcf"

```

Count PASS variants:

```
bcftools view -H -f PASS "$OUT/2_variant_call/OC_blood.all.VQSR.vcf" | wc -l
```

8. Functional annotation with ANNOVAR

"Somewhere, something incredible is waiting to be known."

— Carl Sagan

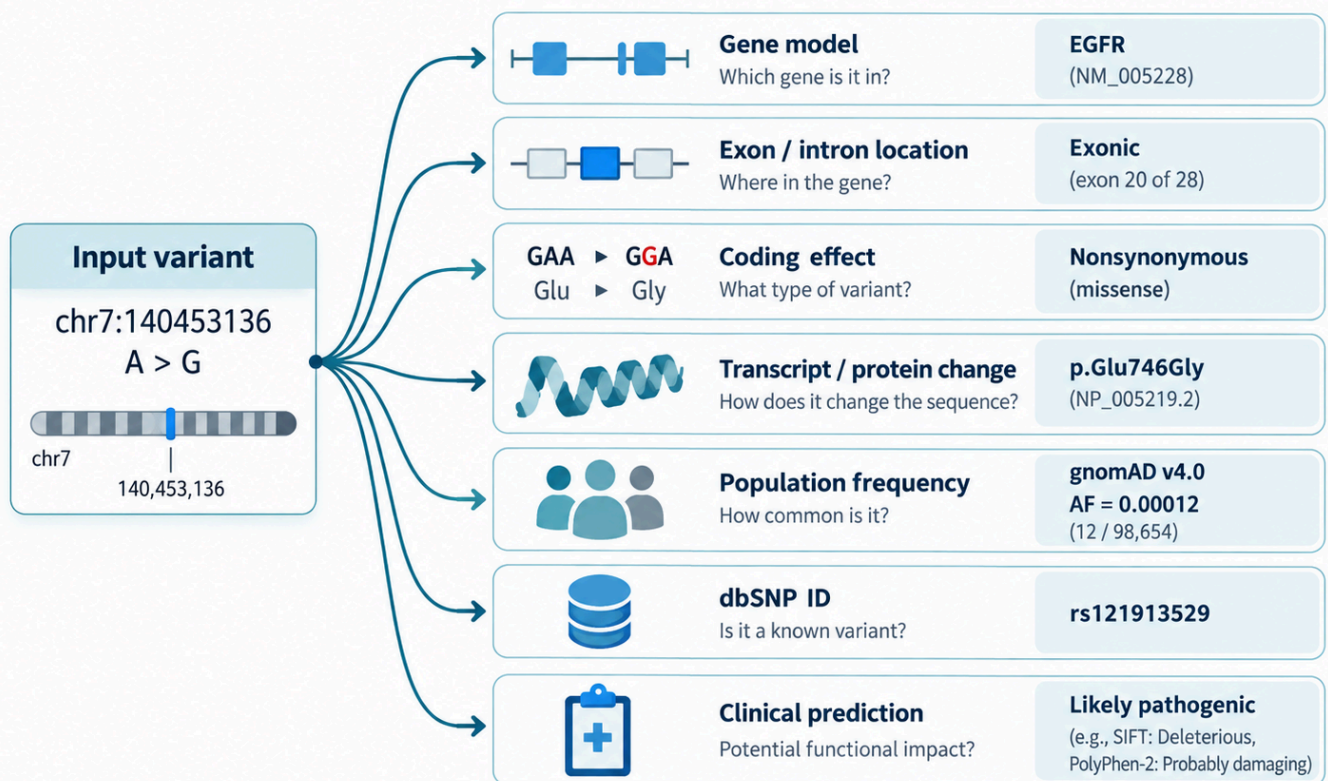
VQSR tells us whether a variant is likely real. ANNOVAR tells us what it may do.

ANNOVAR can annotate:

1. Gene and exon/intron/intergenic location.
2. Coding effect: synonymous, nonsynonymous, stopgain, stoploss, frameshift.
3. Transcript and protein change.
4. Population frequency, for example gnomAD or 1000 Genomes.
5. dbSNP identifier.
6. Clinical or functional prediction databases.

Functional annotation with ANNOVAR

One variant. Multiple layers of biological interpretation.



ANNOVAR integrates multiple biological databases to functionally annotate genetic variants.

From a single variant to a comprehensive biological and clinical interpretation.

8.1 Convert VCF to ANNOVAR input

```
mkdir -p "$OUT/3_annotation"

perl "$ANNOVAR/convert2annovar.pl" \
  -format vcf4old \
  "$OUT/2_variant_call/OC_blood.all.VQSR.vcf" \
  -includeinfo \
  -comment \
  -outfile "$OUT/3_annotation/OC_blood.all.avinput"
```

8.2 Gene annotation

```
perl "$ANNOVAR/annotate_variation.pl" \
  -geneanno \
  -buildver hg38 \
  -dbtype refGeneWithVer \
  "$OUT/3_annotation/OC_blood.all.avinput" \
  "$HUMANDB"
```

8.3 Multi-annotation table

```
perl "$ANNOVAR/table_annovar.pl" \
  "$OUT/3_annotation/OC_blood.all.avinput" \
  "$HUMANDB" \
  -buildver hg38 \
  -out "$OUT/3_annotation/OC_blood_anno" \
  -remove \
  -protocol refGeneWithVer \
  -operation g
```

If frequency and prediction databases are available, a richer protocol is:

```
perl "$ANNOVAR/table_annovar.pl" \
  "$OUT/3_annotation/OC_blood.all.avinput" \
  "$HUMANDB" \
  -buildver hg38 \
  -out "$OUT/3_annotation/OC_blood_anno" \
  -remove \
  -protocol refGeneWithVer,cytoBand,dbnsfp30a,exac03,gnomad_genome,avsnp147 \
  -operation g,r,f,f,f,f
```

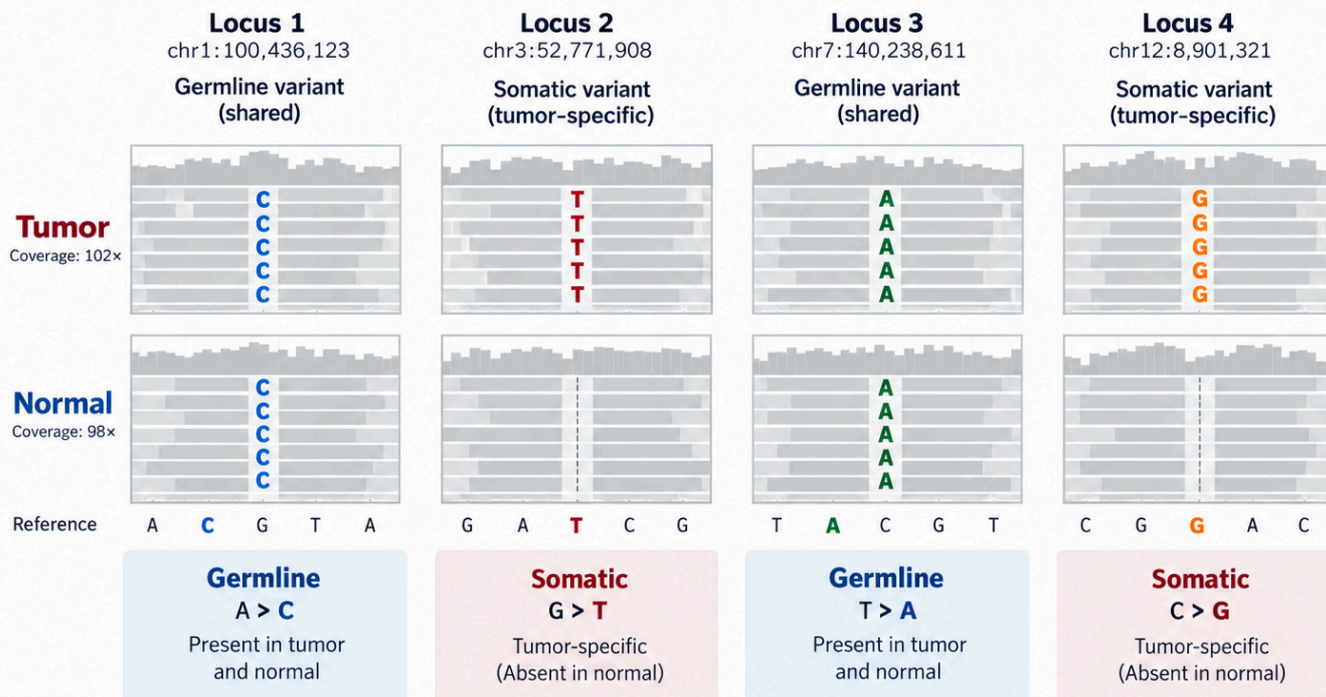
9. Somatic TMB using matched normal

The germline/joint VCF is not a formal somatic TMB result. Formal TMB should use tumor-normal somatic calling.

TMB is usually:

TMB = number of qualifying somatic variants / callable target size in megabases

Tumor-normal TMB



$$\text{TMB} = \frac{\text{Number of qualifying somatic variants}}{\text{Callable target size (Mb)}} = \frac{N}{\text{Mb}}$$

Matched normal helps exclude germline variants

By comparing tumor and normal from the same individual, shared (germline) variants can be removed, leaving somatic variants for TMB calculation.

9.1 Mutect2 tumor-normal calling

```
TUMOR=$OUT/1_alignment/OC/OC_sorted.markdup.BQSR.bam
NORMAL=$OUT/1_alignment/PBMC/blood_sorted.markdup.BQSR.bam
SOMOUT=$OUT/5_somatic_tmb/
mkdir -p "$SOMOUT"

gatk --java-options "-Xmx16g -XX:ParallelGCThreads=4" Mutect2 \
  -R "$REFERENCE" \
  -I "$TUMOR" \
  -I "$NORMAL" \
  --normal-sample blood \
  -L "$TARGETS" \
  --germline-resource "$HIGHCONF" \
  -O "$SOMOUT/somatic.raw.vcf.gz" \
  --native-pair-hmm-threads 8 \
  --f1r2-tar-gz "$SOMOUT/f1r2.tar.gz"
```

9.2 Orientation bias model

```
gatk --java-options "-Xmx8g -XX:ParallelGCThreads=4" LearnReadOrientationModel \
-I "$SOMOUT/f1r2.tar.gz" \
-O "$SOMOUT/orientation_model.tar.gz"

gatk --java-options "-Xmx12g -XX:ParallelGCThreads=4" FilterMutectCalls \
-R "$REFERENCE" \
-V "$SOMOUT/somatic.raw.vcf.gz" \
--stats "$SOMOUT/somatic.raw.vcf.gz.stats" \
--ob-priors "$SOMOUT/orientation_model.tar.gz" \
-O "$SOMOUT/somatic.filtered.vcf.gz"

bcftools view -f PASS \
-Oz \
-o "$SOMOUT/somatic.pass.vcf.gz" \
"$SOMOUT/somatic.filtered.vcf.gz"

bcftools index -t "$SOMOUT/somatic.pass.vcf.gz"
```

9.3 Compute TMB

```
TARGET_SIZE_MB=$(awk 'BEGIN{FS="\t"} {sum += ($3 - $2)} END {printf "%.6f", sum / 1000000}'
"$TARGETS")
PASS_VARIANTS=$(bcftools view -H -f PASS "$SOMOUT/somatic.filtered.vcf.gz" | wc -l)

awk -v n="$PASS_VARIANTS" -v mb="$TARGET_SIZE_MB" \
'BEGIN {printf "somatic TMB = %.6f variants/Mb\n", n / mb}'
```

Take-home question: Why we don't use the joint VCF we generated in 7.3 for TMB calculation? What is the difference between these two variants?

10. CNV detection with CNVkit

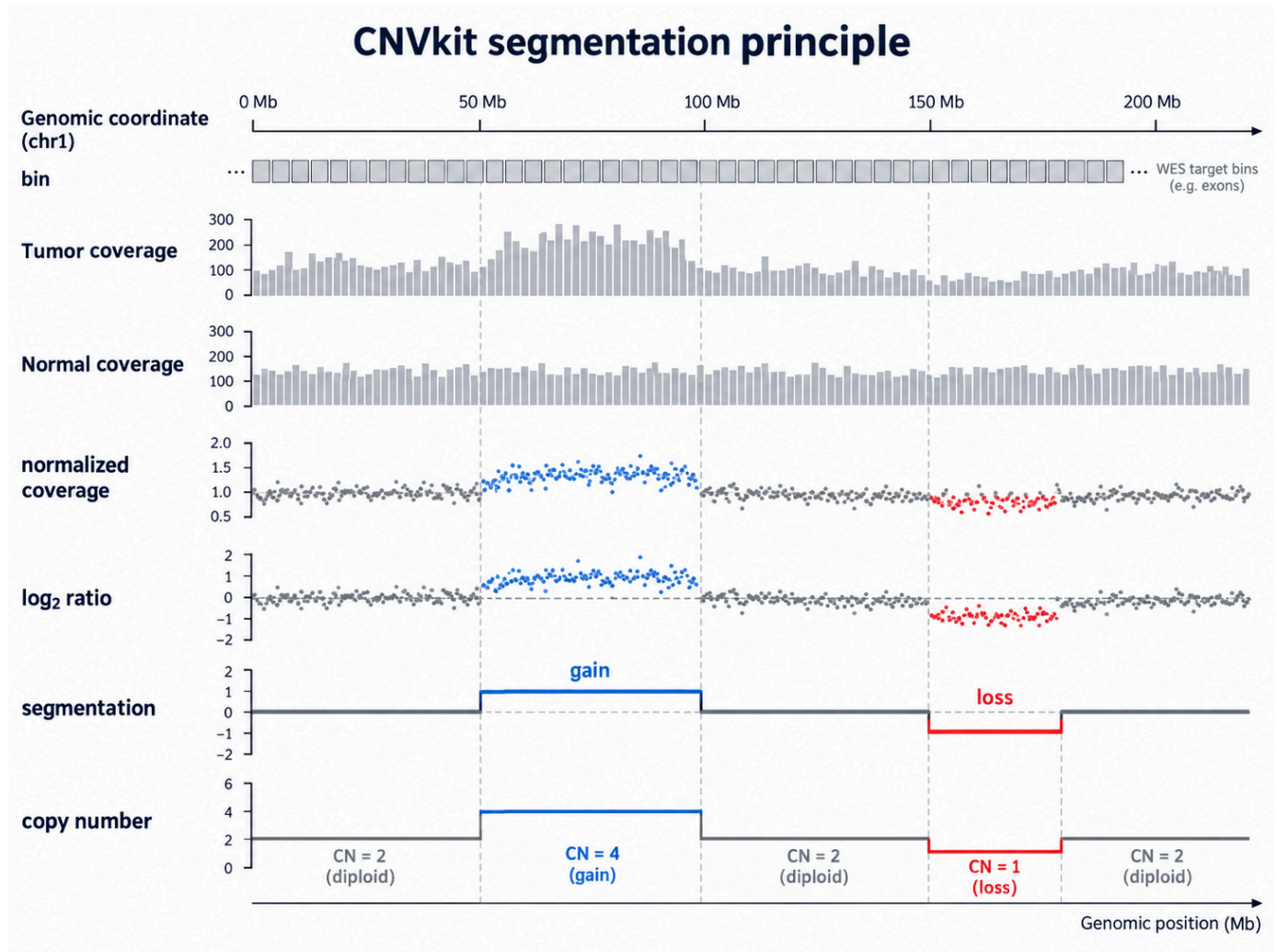
CNVkit detects copy-number changes using binned read depth. In WES, target bins correspond to capture intervals. In hybrid capture mode, off-target bins can also be used; in amplicon mode, only target bins are used.

The CNVkit logic is:

1. Divide the genome or target space into bins.
2. Compute depth per bin in tumor and normal.
3. Normalize tumor depth against normal depth and systematic biases.
4. Calculate a log₂ ratio per bin.
5. Merge neighboring bins into segments.

6. Convert segment log2 ratios into integer copy numbers.

The default CNVkit segmentation method is CBS, circular binary segmentation. CBS tests whether adjacent bins should be split into two segments with different mean log2 ratios.



10.1 Prepare target BED

```
CNVOUT=$OUT/4_cnv
mkdir -p "$CNVOUT"
ln -s "$TARGETS" "$CNVOUT/targets.bed"
```

10.2 Run CNVkit batch

```
cnvkit.py batch \
  "$TUMOR" \
  -n "$NORMAL" \
  -f "$REFERENCE" \
  -t "$CNVOUT/targets.bed" \
  -m amplicon \
  --output-reference "$CNVOUT/reference.cnn" \
  -d "$CNVOUT/cnvkit_out" \
  --scatter \
```

```
--diagram \  
-p 8
```

Important outputs:

File	Level	Meaning
<code>.cnr</code>	bin	Normalized bin-level log2 ratios
<code>.cns</code>	segment	Segmented log2 ratios
<code>.call.cns</code>	segment	Absolute copy-number calls
<code>scatter.png</code>	genome	Genome-wide log2 ratio plot
<code>diagram.pdf</code>	chromosome	Chromosome-level CNV summary

10.3 Interpret `.call.cns`

Key columns:

```
chromosome start end gene log2 cn depth p_ttest probes weight
```

Interpretation:

Column	Meaning
<code>log2</code>	Normalized segment log2 ratio
<code>cn</code>	Called integer copy number
<code>probes</code>	Number of supporting bins
<code>weight</code>	Segment confidence
<code>p_ttest</code>	Statistical significance of the segment

The `gene` column is inherited from the fourth BED column. The shared target BED has only chromosome, start, and end columns, so map intervals to genes before making gene-level claims; do not treat this column as a normalized gene-name list.

11. Interpretation checklist

Before interpreting a WES result, check:

1. Is the reference genome consistent across all resources?
2. Are sample names correct in read groups?

3. Are BAMs coordinate-sorted and indexed?
4. Are duplicate metrics reasonable?
5. Is coverage sufficient over the capture targets?
6. Does the callset contain a plausible Ti/Tv ratio?
7. Are VQSR resources in the same genome build?
8. For TMB, was a matched normal used?
9. Are annotations in the same transcript and genome version?

12. File summary

Output	Meaning
<code>*.clean.fastq.gz</code>	QC-passed reads
<code>*.sam</code> , <code>*.bam</code>	Alignments before sorting
<code>*_sorted.bam</code>	Coordinate-sorted alignments
<code>*_sorted.markdup.bam</code>	Alignment with PCR duplicates marked
<code>*_sorted.markdup.BQSR.bam</code>	Final BAM used for variant calling
<code>*.g.vcf</code>	GVCF with variants and hom-ref blocks
<code>combined.g.vcf</code>	Multi-sample GVCF
<code>*_variants.vcf</code>	Joint-genotyped VCF
<code>*.snps.VQSR.vcf</code>	SNP VQSR output
<code>*.indel.VQSR.vcf</code>	Indel VQSR output
<code>*.all.VQSR.vcf</code>	Merged SNP/indel callset
<code>*.hg38_multianno.txt</code>	ANNOVAR annotated table
<code>somatic.filtered.vcf.gz</code>	Tumor-normal somatic calls
<code>call.cns</code>	CNVkit copy-number segments

13. Summary

"Progress in science depends on new techniques, new discoveries and new ideas, probably in that order."
— Sydney Brenner

WES analysis converts uncertain sequencing observations into increasingly interpretable biological statements: first reads become alignments, alignments become variants, variants become statistically filtered calls, and calls finally become annotated biological hypotheses. Whole-exome sequencing gives us a practical way to read one of the most information-rich parts of that book—the protein-coding exome, and so we can say: we celebrate the revelation of the draft of the human book of life.

Any questions are welcome and please contact me at jiahaoma@stu.pku.edu.cn.

“纸上得来终觉浅，绝知此事要躬行。”